



Feature Request 02: The Memoir Owner Dashboard

Submission Date: Wednesday, 26th August 2026

Pre-requisites

Before starting this task, make sure you have:

- Feature Request 01 merged and working. This task begins where that one ended: an owner with an account and a set of answers sitting in the database.
- A working understanding of the Git workflow: branching, pull requests, review, and merge.

Description

What is the Owner Dashboard?

The dashboard is the workspace the memoir owner returns to between the moment they create a memoir and the moment they publish it. It is where memories accumulate. Everything the family will eventually read passes through this screen first.

It is not an admin panel. The owner is a person who recently lost a loved one, opening this on a Tuesday evening. The screen should feel like opening a book that is being written, not like a content management system.



Task

Build the memoir owner dashboard and the capture flows that feed it.

First, create the dashboard itself. The owner can add memories to it, and see everything they have added so far. Three ways to add a memory:

- **Written entry.** Typed text.
- **Voice recording.** Captured in the browser, on both mobile and desktop, with pause and re-record.
- **Photograph.** Uploaded, with an optional caption.

A single memory may combine these. Somebody who records ninety seconds about a bakery and attaches the photograph of it has made one memory, not two. Do not build a schema that forces them apart.

Storage rule, non-negotiable: audio and image files do not go in the database and do not stream through your API server. They go to object storage directly from the browser using a presigned URL your API issues. The database stores the key, the size, the type, and who uploaded it.

Save and resume. A half-written memory must survive a closed tab. Note that server-side draft state and browser-side draft state are different things with different guarantees, and the difference matters: **localStorage** protects against a browser crash, and nothing else. It does not protect against the owner opening the memoir on their phone.

Editing and deletion, before publication only. The memoir becomes immutable when it is published, which is a later cycle. Until then the owner can correct a typo and remove something they added by mistake. Locking a memory the moment it is saved is a misreading of the constraint, and it will lose contributions.

In scope: the owner's path, from the end of onboarding through to a dashboard holding real memories of all three kinds, persisted and still there after logout.

Out of scope: contributors, invite links, and anybody who is not the owner. Transcription of recordings. AI organization into chapters. Publishing, the published webpage, and the PDF. Comments and reactions. Video.

Out of scope means do not build it. It does not mean do not think about it: the audio you store this cycle is the audio a transcription job reads next cycle, so store it in a form that survives.



Deliverables

1. A short feature specification: what a memory is, what the capture flows are, what happens on each failure you expect, and what "done" means. One or two pages. Write it before you write code.
2. A database migration adding only the tables this feature uses.
3. Working implementation: dashboard, all three capture types, media in object storage, drafts that survive.
4. A repository with a real commit history.

All work is delivered through GitHub. Every work item is built on its own branch and submitted as a pull request, i.e. no direct commits to the main branch. Pull requests must describe what was built and why, and must be reviewed before being merged.

The commit history is part of the deliverable. A repository whose work arrives in one commit at the end has not met this requirement, regardless of what it contains.

Definition of Done

The task is done when the owner of a memoir can:

1. Land on a dashboard after onboarding that shows the subject's name and years, and recognise it as the same product they just came through.
2. Open a memoir with nothing in it and see something intentional, not an empty list.
3. Write an entry, save it, and see it in the feed.
4. Record their voice, pause, re-record, save, and play it back afterwards from storage rather than from browser memory.
5. Upload a photograph with a caption and see it in the feed.
6. Combine a recording and a photograph into one memory rather than two.
7. Leave a half-finished memory, close the tab, come back, and find it waiting.
8. Correct a typo in something they saved, and remove something they added by mistake.
9. Log out, log back in on a different device, and find everything exactly as they left it.
10. Lose their connection mid-upload and get a message a non-technical person understands, without silently losing what they wrote.



Work Allocation

Group members claim work rather than being assigned it. Service and data work is claimed by AI Backend students; screens and client-side work by Full-Stack students.

The seam this cycle runs through the upload: the browser asks the API for permission to upload, uploads directly to storage, then tells the API it is done. Three parties, two of them yours. Agree the shape of that exchange between the two tracks before either side starts building, and write it down.

Recommended Resources/Tools

- The Memoir Project PRD - Memory Capture, Constraints, and the Design section
- schema.sql - memory, media_asset, memory_media, transcript
- Task 03 system architecture output
- Use Supabase object storage